BigQuery Commands

Data Engineer Handbook

Ready-to-use bq CLI, GoogleSQL (DDL / DML / scripting), loading and exporting, partitioning, scheduling, IAM, monitoring and cost control.

Conventions used: `PROJECT`, `DATASET`, `TABLE`, `BUCKET` are placeholders. Replace them with your own values. Commands were written for the current bq CLI and GoogleSQL.

Contents

1. Setup and Authentication 3

2. Datasets 3

3. Tables: Create, Inspect, Copy, Delete 3

4. Loading Data (bq load / LOAD DATA) 5

5. Exporting Data 5

6. Running Queries (bq query) 6

7. Job Management 6

8. DML: INSERT / UPDATE / DELETE / MERGE 6

9. Partitioning and Clustering 7

10. Views, Materialized Views, Routines 7

11. Scheduled Queries and Scripting 7

12. External Tables and Federated Queries 8

13. Time Travel, Snapshots, Clones, Recovery 8

14. Access Control (IAM) 9

15. Monitoring with INFORMATION_SCHEMA 9

16. Essential SQL Patterns for Pipelines 10

17. Python Client (google-cloud-bigquery) 11

18. Cost and Performance Checklist 11

19. Quick Reference: bq Command Cheat Sheet 12

1. Setup and Authentication

```
# Install / update the Google Cloud SDK (bq is bundled), then log in
gcloud auth login
gcloud auth application-default login      # for Python / client libraries
gcloud config set project PROJECT_ID
gcloud config list

# Service account auth (CI/CD)
gcloud auth activate-service-account --key-file=sa-key.json

# Default location / project for bq (~/.bigqueryrc)
bq init
bq version
bq help query                              # help for any command
```

Global flags you will use often

Flag	Meaning
--project_id=PROJECT	Run the command against a specific project
--location=US | asia-south1	Job location (must match dataset location)
--format=prettyjson | json | csv | pretty	Output format
--use_legacy_sql=false	Use GoogleSQL (standard SQL). Always set this
--nosync	Run async: return job id immediately

2. Datasets

```
bq ls                                      # list datasets in default project
bq ls --project_id=PROJECT                 # list datasets in another project
bq ls -a                                   # include hidden datasets

# Create dataset
bq mk --dataset --location=asia-south1 \
      --description="Raw layer" --label=env:prod PROJECT:raw

# Default table expiration (seconds) and partition expiration
bq mk --dataset --default_table_expiration=2592000 PROJECT:staging

bq show --format=prettyjson PROJECT:raw         # metadata
bq update --description="New desc" PROJECT:raw
bq update --set_label team:data PROJECT:raw

bq rm -r -d PROJECT:staging                     # delete dataset + all tables (asks to confirm)
bq rm -r -f -d PROJECT:staging                  # no prompt
```

Same in SQL

```
CREATE SCHEMA IF NOT EXISTS `PROJECT.raw`
OPTIONS (location = 'asia-south1', description = 'Raw layer',
         default_table_expiration_days = 30);

ALTER SCHEMA `PROJECT.raw` SET OPTIONS (description = 'Updated');
DROP SCHEMA IF EXISTS `PROJECT.staging` CASCADE;
```

3. Tables: Create, Inspect, Copy, Delete

```
bq ls PROJECT:DATASET                      # list tables
bq ls --max_results=1000 DATASET
bq show DATASET.TABLE                      # schema, rows, size
bq show --schema --format=prettyjson DATASET.TABLE > schema.json
bq head -n 10 DATASET.TABLE                # preview rows (free)
bq head --selected_fields=id,name -n 5 DATASET.TABLE

# Create empty table with inline schema
```

```
bq mk --table DATASET.customers id:INT64,name:STRING,created_at:TIMESTAMP

# Create from schema file, partitioned + clustered
bq mk --table \
  --time_partitioning_type=DAY --time_partitioning_field=created_at \
  --time_partitioning_expiration=7776000 \
  --require_partition_filter=true \
  --clustering_fields=customer_id,country \
  --description="Customer events" --label=layer:silver \
  DATASET.events ./schema.json

# Copy
bq cp DATASET.src DATASET.dst                  # fails if dst exists
bq cp -f DATASET.src DATASET.dst               # overwrite
bq cp -a DATASET.src DATASET.dst               # append
bq cp -n DATASET.src DATASET.dst               # no-clobber

# Update metadata
bq update --expiration 86400 DATASET.TABLE     # expire in 1 day (0 = never)
bq update --description "text" DATASET.TABLE
bq update --schema schema.json DATASET.TABLE   # add NULLABLE columns / relax REQUIRED

# Delete
bq rm -t -f DATASET.TABLE
bq rm -f -t 'DATASET.events$20260101'          # delete one partition
```

schema.json format

```
[
  {"name": "id",         "type": "INT64",     "mode": "REQUIRED"},
  {"name": "name",       "type": "STRING",    "mode": "NULLABLE"},
  {"name": "created_at", "type": "TIMESTAMP", "mode": "NULLABLE"},
  {"name": "tags",       "type": "STRING",    "mode": "REPEATED"},
  {"name": "address",    "type": "RECORD",    "mode": "NULLABLE",
   "fields": [{"name": "city", "type": "STRING"}, {"name": "pin", "type": "STRING"}]}
]
```

Create / alter in SQL

```
CREATE TABLE IF NOT EXISTS `PROJECT.silver.events` (
  event_id   STRING NOT NULL,
  customer_id INT64,
  event_ts   TIMESTAMP,
  payload    JSON,
  tags       ARRAY<STRING>
)
PARTITION BY DATE(event_ts)
CLUSTER BY customer_id
OPTIONS (partition_expiration_days = 365,
         require_partition_filter = TRUE,
         description = 'Silver events');

-- CTAS
CREATE OR REPLACE TABLE silver.orders_2026
PARTITION BY order_date CLUSTER BY customer_id AS
SELECT * FROM raw.orders WHERE order_date >= '2026-01-01';

-- Schema changes
ALTER TABLE silver.events ADD COLUMN IF NOT EXISTS source STRING;
ALTER TABLE silver.events DROP COLUMN IF EXISTS source;
ALTER TABLE silver.events RENAME COLUMN payload TO raw_payload;
ALTER TABLE silver.events ALTER COLUMN customer_id DROP NOT NULL;
ALTER TABLE silver.events ALTER COLUMN tags SET OPTIONS (description = 'event tags');
ALTER TABLE silver.events SET OPTIONS (expiration_timestamp = TIMESTAMP '2027-01-01');
ALTER TABLE silver.events RENAME TO events_v2;

DROP TABLE IF EXISTS silver.tmp_events;
TRUNCATE TABLE silver.tmp_events;
```

4. Loading Data (bq load / LOAD DATA)

```
# CSV with header, autodetect schema, from GCS
bq load --source_format=CSV --skip_leading_rows=1 --autodetect \
  DATASET.sales gs://BUCKET/sales/2026-01-*.csv

# CSV with explicit schema, custom delimiter, tolerate bad rows
bq load --source_format=CSV --skip_leading_rows=1 \
  --field_delimiter='|' --allow_quoted_newlines --max_bad_records=10 \
  --null_marker='NA' DATASET.sales gs://BUCKET/sales.psv ./schema.json

# Newline-delimited JSON
bq load --source_format=NEWLINE_DELIMITED_JSON --autodetect \
  DATASET.logs gs://BUCKET/logs/*.json

# Parquet / Avro / ORC (schema is read from the files)
bq load --source_format=PARQUET DATASET.facts gs://BUCKET/facts/*.parquet

# From a local file
bq load --source_format=CSV --skip_leading_rows=1 --autodetect DATASET.t ./local.csv

# Write dispositions
bq load --replace  ...                        # WRITE_TRUNCATE
bq load --noreplace ...                       # WRITE_APPEND (default for existing table)

# Schema evolution during load
bq load --schema_update_option=ALLOW_FIELD_ADDITION \
        --schema_update_option=ALLOW_FIELD_RELAXATION ...

# Load into a single partition
bq load --replace --source_format=CSV 'DATASET.events$20260115' gs://BUCKET/d1.csv

# Partitioned + clustered on load
bq load --source_format=PARQUET \
  --time_partitioning_field=event_date --clustering_fields=customer_id \
  DATASET.events gs://BUCKET/events/*.parquet

# Hive-partitioned folders (gs://BUCKET/events/dt=2026-01-01/...)
bq load --source_format=PARQUET --hive_partitioning_mode=AUTO \
  --hive_partitioning_source_uri_prefix=gs://BUCKET/events/ \
  DATASET.events gs://BUCKET/events/*

# Run async and check later
bq --nosync load ... ; bq wait JOB_ID
```

SQL equivalent

```
LOAD DATA INTO raw.sales
FROM FILES (format = 'CSV', uris = ['gs://BUCKET/sales/*.csv'],
            skip_leading_rows = 1);

LOAD DATA OVERWRITE raw.facts
FROM FILES (format = 'PARQUET', uris = ['gs://BUCKET/facts/*.parquet']);
```

5. Exporting Data

```
# Table to GCS (use wildcard for tables > 1 GB)
bq extract --destination_format=CSV --compression=GZIP --print_header=true \
  DATASET.sales 'gs://BUCKET/export/sales-*.csv.gz'

bq extract --destination_format=NEWLINE_DELIMITED_JSON DATASET.logs 'gs://BUCKET/logs-*.json'
bq extract --destination_format=PARQUET DATASET.facts 'gs://BUCKET/facts-*.parquet'
bq extract --destination_format=AVRO --use_avro_logical_types DATASET.facts 'gs://BUCKET/f-*.avro'

-- Export a query result
EXPORT DATA OPTIONS (
  uri = 'gs://BUCKET/out/orders-*.parquet',
  format = 'PARQUET', overwrite = true
) AS
```

```
SELECT * FROM silver.orders WHERE order_date = CURRENT_DATE();
```

6. Running Queries (bq query)

```
bq query --use_legacy_sql=false 'SELECT COUNT(*) FROM `PROJECT.DATASET.TABLE`'

# Dry run: shows bytes that would be processed (no cost)
bq query --use_legacy_sql=false --dry_run 'SELECT * FROM DATASET.big_table'

# Cap the cost: query fails if it would scan more than 10 GB
bq query --use_legacy_sql=false --maximum_bytes_billed=10737418240 'SELECT ...'

# Save result to a table
bq query --use_legacy_sql=false --destination_table=DATASET.result \
  --replace 'SELECT ...'
bq query --use_legacy_sql=false --destination_table=DATASET.result \
  --append_table 'SELECT ...'

# Output formats / rows
bq query --use_legacy_sql=false --format=csv --max_rows=100000 'SELECT ...' > out.csv

# Query from file
bq query --use_legacy_sql=false < query.sql

# Query parameters
bq query --use_legacy_sql=false \
  --parameter=start_date:DATE:2026-01-01 \
  --parameter=min_amt:INT64:100 \
  'SELECT * FROM sales WHERE dt >= @start_date AND amount >= @min_amt'

# Batch priority (cheaper slots queueing), job labels
bq query --use_legacy_sql=false --batch --label=team:etl 'SELECT ...'

# Run in a specific location / project
bq --project_id=PROJECT --location=asia-south1 query --use_legacy_sql=false 'SELECT 1'
```

7. Job Management

```
bq ls -j -a -n 20                              # last 20 jobs, all users
bq ls -j --min_creation_time=1767225600000     # since epoch ms
bq show -j JOB_ID                              # status, bytes, errors
bq show --format=prettyjson -j JOB_ID
bq cancel JOB_ID
bq wait JOB_ID 300                             # wait up to 300 seconds
```

8. DML: INSERT / UPDATE / DELETE / MERGE

```
INSERT INTO silver.customers (id, name, city)
VALUES (1, 'Asha', 'Pune'), (2, 'Ravi', 'Hyderabad');

INSERT INTO silver.customers SELECT * FROM staging.customers_new;

UPDATE silver.customers SET city = 'Pune' WHERE id = 2;
DELETE FROM silver.customers WHERE is_deleted = TRUE;

-- UPSERT (the core of incremental loads)
MERGE silver.customers T
USING staging.customers_new S
ON T.id = S.id
WHEN MATCHED AND S.updated_at > T.updated_at THEN
  UPDATE SET name = S.name, city = S.city, updated_at = S.updated_at
WHEN NOT MATCHED THEN
  INSERT (id, name, city, updated_at) VALUES (S.id, S.name, S.city, S.updated_at)
WHEN NOT MATCHED BY SOURCE THEN
  DELETE;                                      -- optional: remove rows missing from source

-- Replace one partition atomically
DELETE FROM silver.events WHERE DATE(event_ts) = '2026-01-15';
```

```
INSERT INTO silver.events SELECT * FROM staging.events WHERE DATE(event_ts) = '2026-01-15';
```

Tip: Always filter the target of MERGE/UPDATE/DELETE on the partition column to avoid full-table scans.

9. Partitioning and Clustering

```
-- Time-unit column partition (DAY | HOUR | MONTH | YEAR)
CREATE TABLE ds.t (ts TIMESTAMP, id INT64)
PARTITION BY TIMESTAMP_TRUNC(ts, MONTH);

-- Ingestion-time partition (pseudo column _PARTITIONTIME / _PARTITIONDATE)
CREATE TABLE ds.t2 (id INT64) PARTITION BY _PARTITIONDATE;

-- Integer-range partition
CREATE TABLE ds.t3 (cust_id INT64, v STRING)
PARTITION BY RANGE_BUCKET(cust_id, GENERATE_ARRAY(0, 1000000, 10000));

-- Cluster (up to 4 columns, order matters)
CREATE TABLE ds.t4 (dt DATE, country STRING, cust_id INT64)
PARTITION BY dt CLUSTER BY country, cust_id;

-- Inspect partitions
SELECT partition_id, total_rows, total_logical_bytes, last_modified_time
FROM DATASET.INFORMATION_SCHEMA.PARTITIONS
WHERE table_name = 'events' ORDER BY partition_id DESC;

-- Make a table require a partition filter
ALTER TABLE ds.t SET OPTIONS (require_partition_filter = TRUE);

# CLI: list partitions of a table
bq ls --format=prettyjson 'DATASET.events$__PARTITIONS_SUMMARY__'
```

10. Views, Materialized Views, Routines

```
CREATE OR REPLACE VIEW gold.v_daily_sales AS
SELECT order_date, SUM(amount) AS revenue FROM silver.orders GROUP BY order_date;

CREATE MATERIALIZED VIEW gold.mv_daily_sales
OPTIONS (enable_refresh = TRUE, refresh_interval_minutes = 30) AS
SELECT order_date, COUNT(*) AS orders, SUM(amount) AS revenue
FROM silver.orders GROUP BY order_date;

-- SQL UDF
CREATE OR REPLACE FUNCTION utils.clean_phone(p STRING) AS (
  REGEXP_REPLACE(p, r'[^0-9]', '')
);

-- Stored procedure
CREATE OR REPLACE PROCEDURE etl.load_daily(run_date DATE)
BEGIN
  DELETE FROM silver.orders WHERE order_date = run_date;
  INSERT INTO silver.orders SELECT * FROM raw.orders WHERE order_date = run_date;
END;
CALL etl.load_daily(CURRENT_DATE());

-- Table function
CREATE OR REPLACE TABLE FUNCTION utils.orders_by_day(d DATE) AS
SELECT * FROM silver.orders WHERE order_date = d;

bq ls --routines DATASET
bq show --routine DATASET.load_daily
bq rm -f --routine DATASET.load_daily
```

11. Scheduled Queries and Scripting

```
# Scheduled query via CLI (BigQuery Data Transfer Service)
bq mk --transfer_config \
  --project_id=PROJECT --location=asia-south1 \
```

```
  --target_dataset=gold \
  --display_name='daily_sales_refresh' \
  --data_source=scheduled_query \
  --schedule='every 24 hours' \
  --params='{"query":"SELECT CURRENT_DATE() d, COUNT(*) c FROM silver.orders",
             "destination_table_name_template":"daily_counts",
             "write_disposition":"WRITE_APPEND"}'

bq ls --transfer_config --transfer_location=asia-south1
bq rm --transfer_config projects/PROJECT_NUM/locations/asia-south1/transferConfigs/ID

-- Multi-statement script with variables, loops, control flow, error handling
DECLARE run_date DATE DEFAULT CURRENT_DATE();
DECLARE cnt INT64;

BEGIN
  SET cnt = (SELECT COUNT(*) FROM raw.orders WHERE order_date = run_date);
  IF cnt = 0 THEN
    RAISE USING MESSAGE = 'No data for ' || CAST(run_date AS STRING);
  END IF;
  MERGE silver.orders T USING raw.orders S ON T.order_id = S.order_id
  WHEN MATCHED THEN UPDATE SET amount = S.amount
  WHEN NOT MATCHED THEN INSERT ROW;
EXCEPTION WHEN ERROR THEN
  SELECT @@error.message, @@error.statement_text;
END;

-- Transaction
BEGIN TRANSACTION;
  DELETE FROM silver.orders WHERE order_date = '2026-01-15';
  INSERT INTO silver.orders SELECT * FROM staging.orders;
COMMIT TRANSACTION;                          -- or ROLLBACK TRANSACTION;

-- Dynamic SQL
EXECUTE IMMEDIATE FORMAT('SELECT COUNT(*) FROM `%s.%s`', 'silver', 'orders');
```

12. External Tables and Federated Queries

```
# CLI: external table over GCS files
bq mkdef --source_format=PARQUET 'gs://BUCKET/facts/*.parquet' > ext_def.json
bq mk --table --external_table_definition=ext_def.json DATASET.ext_facts

bq mkdef --source_format=CSV --autodetect=true 'gs://BUCKET/x/*.csv' > csv_def.json

-- SQL external table
CREATE OR REPLACE EXTERNAL TABLE raw.ext_events
OPTIONS (format = 'PARQUET', uris = ['gs://BUCKET/events/*.parquet']);

-- Hive-partitioned external table
CREATE EXTERNAL TABLE raw.ext_hive
WITH PARTITION COLUMNS
OPTIONS (format = 'PARQUET', uris = ['gs://BUCKET/events/*'],
         hive_partition_uri_prefix = 'gs://BUCKET/events');

-- Federated query to Cloud SQL / Spanner
SELECT * FROM EXTERNAL_QUERY('PROJECT.asia-south1.my_connection',
                             'SELECT id, name FROM customers WHERE active = 1');
```

13. Time Travel, Snapshots, Clones, Recovery

```
-- Query data as it was (default window: 7 days, configurable 2-7)
SELECT * FROM silver.orders
FOR SYSTEM_TIME AS OF TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 2 HOUR);

-- Restore a table / recover from a bad update
CREATE OR REPLACE TABLE silver.orders_restored AS
SELECT * FROM silver.orders
FOR SYSTEM_TIME AS OF TIMESTAMP '2026-01-15 10:00:00+05:30';
```

```
-- Snapshot (cheap, read-only, point-in-time) and clone (writable)
CREATE SNAPSHOT TABLE backup.orders_20260115 CLONE silver.orders
OPTIONS (expiration_timestamp = TIMESTAMP_ADD(CURRENT_TIMESTAMP(), INTERVAL 90 DAY));
CREATE TABLE dev.orders_clone CLONE prod.orders;

-- Set the time-travel window on a dataset
ALTER SCHEMA silver SET OPTIONS (max_time_travel_hours = 48);

# CLI snapshot and restore from snapshot
bq cp --snapshot --no_clobber --expiration=7776000 DATASET.orders backup.orders_snap
bq cp --restore --no_clobber backup.orders_snap DATASET.orders_restored

# Undelete a dropped table within the window (relative decorator, ms)
bq cp DATASET.orders@-3600000 DATASET.orders_recovered
```

14. Access Control (IAM)

```
# Table / dataset level IAM via CLI
bq add-iam-policy-binding --member='user:name@company.com' \
  --role='roles/bigquery.dataViewer' PROJECT:DATASET.TABLE
bq get-iam-policy PROJECT:DATASET.TABLE
bq remove-iam-policy-binding --member='user:name@company.com' \
  --role='roles/bigquery.dataViewer' PROJECT:DATASET.TABLE

# Project level
gcloud projects add-iam-policy-binding PROJECT \
  --member='serviceAccount:etl@PROJECT.iam.gserviceaccount.com' \
  --role='roles/bigquery.jobUser'

-- SQL grants
GRANT `roles/bigquery.dataEditor` ON SCHEMA `PROJECT.silver`
  TO 'group:data-eng@company.com';
REVOKE `roles/bigquery.dataEditor` ON SCHEMA `PROJECT.silver`
  FROM 'group:data-eng@company.com';

-- Row-level security
CREATE ROW ACCESS POLICY india_only ON silver.orders
GRANT TO ('group:india-team@company.com') FILTER USING (country = 'IN');
DROP ROW ACCESS POLICY india_only ON silver.orders;
```

Role	Use for
roles/bigquery.dataViewer	Read tables and views
roles/bigquery.dataEditor	Read + write tables in a dataset
roles/bigquery.jobUser	Run queries and load jobs (project level)
roles/bigquery.user	Run jobs + create datasets
roles/bigquery.admin	Full control (use sparingly)

15. Monitoring with INFORMATION_SCHEMA

Use the region qualifier that matches your data, for example `` `region-us` `` or `` `region-asia-south1` ``.

```
-- Top 20 most expensive queries in last 7 days (TB billed)
SELECT user_email, job_id,
       ROUND(total_bytes_billed / POW(1024, 4), 3) AS tb_billed,
       ROUND(total_slot_ms / 1000 / 3600, 2)        AS slot_hours,
       query
FROM `region-asia-south1`.INFORMATION_SCHEMA.JOBS_BY_PROJECT
WHERE creation_time >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 7 DAY)
  AND job_type = 'QUERY' AND state = 'DONE'
ORDER BY total_bytes_billed DESC LIMIT 20;

-- Failed jobs today
SELECT job_id, user_email, error_result.reason, error_result.message
FROM `region-asia-south1`.INFORMATION_SCHEMA.JOBS_BY_PROJECT
```

```
WHERE DATE(creation_time) = CURRENT_DATE() AND error_result IS NOT NULL;

-- Storage per table
SELECT table_schema, table_name,
       ROUND(active_logical_bytes / POW(1024,3), 2) AS active_gb,
       ROUND(long_term_logical_bytes / POW(1024,3), 2) AS long_term_gb
FROM `region-asia-south1`.INFORMATION_SCHEMA.TABLE_STORAGE
ORDER BY active_logical_bytes DESC LIMIT 20;

-- Columns and tables metadata
SELECT table_name, column_name, data_type, is_partitioning_column, clustering_ordinal_position
FROM DATASET.INFORMATION_SCHEMA.COLUMNS WHERE table_name = 'events';

SELECT table_name, table_type, creation_time, ddl
FROM DATASET.INFORMATION_SCHEMA.TABLES;

-- Streaming / load jobs
SELECT * FROM `region-asia-south1`.INFORMATION_SCHEMA.JOBS_BY_PROJECT
WHERE job_type = 'LOAD' ORDER BY creation_time DESC LIMIT 20;
```

View	What it gives you
INFORMATION_SCHEMA.JOBS(_BY_PROJECT / _BY_USER)	Every job: cost, slots, errors, query text
INFORMATION_SCHEMA.TABLES / COLUMNS	Table and column metadata, DDL
INFORMATION_SCHEMA.PARTITIONS	Row count and size per partition
INFORMATION_SCHEMA.TABLE_STORAGE	Active / long-term / time-travel bytes
INFORMATION_SCHEMA.ROUTINES / VIEWS	UDFs, procedures, view definitions
INFORMATION_SCHEMA.SCHEMATA	Datasets, location, options

16. Essential SQL Patterns for Pipelines

De-duplicate (keep latest record per key)

```
SELECT * FROM raw.customers
QUALIFY ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC) = 1;

-- Overwrite table with the de-duplicated result
CREATE OR REPLACE TABLE silver.customers AS
SELECT * FROM raw.customers
QUALIFY ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC) = 1;
```

Window functions, running totals, lag/lead

```
SELECT customer_id, order_date, amount,
  SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date
                    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total,
  LAG(amount)  OVER (PARTITION BY customer_id ORDER BY order_date)    AS prev_amount,
  RANK()       OVER (PARTITION BY customer_id ORDER BY amount DESC)   AS amt_rank
FROM silver.orders;
```

Arrays, structs, JSON

```
SELECT order_id, item.sku, item.qty
FROM silver.orders, UNNEST(items) AS item;                -- flatten ARRAY<STRUCT>

SELECT ARRAY_AGG(STRUCT(sku, qty) ORDER BY sku) AS items FROM t GROUP BY order_id;

SELECT JSON_VALUE(payload, '$.user.id')        AS user_id,
       JSON_QUERY(payload, '$.items')          AS items_json,
       SAFE_CAST(JSON_VALUE(payload,'$.amt') AS NUMERIC) AS amt
FROM raw.events;
```

Dates, safe functions, sampling

```
SELECT DATE_TRUNC(order_date, MONTH), DATE_DIFF(CURRENT_DATE(), order_date, DAY),
       FORMAT_DATE('%Y-%m', order_date), PARSE_DATE('%d/%m/%Y', '15/01/2026'),
       TIMESTAMP(DATETIME '2026-01-15 10:00:00', 'Asia/Kolkata'),
       SAFE_DIVIDE(a, b), SAFE_CAST(x AS INT64), IFNULL(c, 0), COALESCE(a, b, 'n/a')
FROM t;

SELECT * FROM big_table TABLESAMPLE SYSTEM (1 PERCENT);     -- cheap sampling
SELECT * FROM `PROJECT.DATASET.events_*` WHERE _TABLE_SUFFIX BETWEEN '20260101' AND '20260131';
```

Data quality checks

```
-- Null / duplicate / freshness checks
SELECT COUNT(*) AS total,
       COUNTIF(id IS NULL) AS null_ids,
       COUNT(*) - COUNT(DISTINCT id) AS dup_ids,
       MAX(updated_at) AS latest_record,
       TIMESTAMP_DIFF(CURRENT_TIMESTAMP(), MAX(updated_at), HOUR) AS hours_stale
FROM silver.customers;

-- Row-count reconciliation between layers
SELECT 'raw' AS layer, COUNT(*) c FROM raw.orders WHERE order_date = CURRENT_DATE()
UNION ALL
SELECT 'silver', COUNT(*) FROM silver.orders WHERE order_date = CURRENT_DATE();

-- Fail a pipeline when a check breaks
ASSERT (SELECT COUNT(*) FROM silver.orders WHERE amount < 0) = 0
  AS 'Negative amounts found in silver.orders';
```

17. Python Client (google-cloud-bigquery)

```
# pip install google-cloud-bigquery pandas db-dtypes pyarrow
from google.cloud import bigquery
client = bigquery.Client(project="PROJECT")

# Query to DataFrame
df = client.query("SELECT * FROM `PROJECT.ds.t` LIMIT 100").to_dataframe()

# Parameterised query
cfg = bigquery.QueryJobConfig(query_parameters=[
    bigquery.ScalarQueryParameter("d", "DATE", "2026-01-15")])
rows = client.query("SELECT * FROM ds.t WHERE dt = @d", job_config=cfg).result()

# Load from GCS
job = client.load_table_from_uri(
    "gs://BUCKET/sales/*.csv", "PROJECT.ds.sales",
    job_config=bigquery.LoadJobConfig(
        source_format=bigquery.SourceFormat.CSV, skip_leading_rows=1,
        autodetect=True, write_disposition="WRITE_APPEND"))
job.result()

# Load a DataFrame
client.load_table_from_dataframe(df, "PROJECT.ds.t_copy").result()

# Streaming inserts (small volumes; prefer Storage Write API for scale)
errors = client.insert_rows_json("PROJECT.ds.events", [{"id": 1, "name": "a"}])

# Dry run cost estimate
job = client.query("SELECT * FROM ds.big",
                   job_config=bigquery.QueryJobConfig(dry_run=True, use_query_cache=False))
print(f"{job.total_bytes_processed / 1e9:.2f} GB")
```

18. Cost and Performance Checklist

- **Select only the columns you need.** BigQuery is columnar; `SELECT *` bills every column.
- **Partition + cluster** big tables and always filter on the partition column. Set `require_partition_filter = TRUE`.
- **LIMIT does not reduce cost** on its own. Use partition filters, `bq head` or TABLESAMPLE to preview.

- **Dry run first** (`--dry_run`) and set `--maximum_bytes_billed` on ad-hoc and scheduled jobs.
- **Prefer MERGE or partition overwrite** over rewriting whole tables. Avoid many single-row DML statements.
- **Use native load jobs** (free) instead of streaming inserts where latency allows. Parquet / Avro load faster than CSV / JSON.
- **Filter early, join on clustered keys**, and put the largest table first. Pre-aggregate before joining.
- **Avoid cross joins and unbounded self-joins**. Use `APPROX_COUNT_DISTINCT` when exactness is not required.
- **Set table / partition expiration** on staging data, and use long-term storage pricing (untouched 90 days) for archives.
- **Materialized views** for repeated aggregations; BI Engine for dashboards.
- **Check the query plan** in the console (Execution details) for skew, large shuffles and slot time.
- **Label every job** (`--label=team:x`) so cost can be attributed in INFORMATION_SCHEMA.JOBS.

19. Quick Reference: bq Command Cheat Sheet

Task	Command
List datasets / tables	`bq ls` / `bq ls DATASET`
Create dataset	`bq mk --dataset --location=US PROJECT:DATASET`
Create table	`bq mk --table DATASET.T schema.json`
Show metadata / schema	`bq show [--schema] DATASET.T`
Preview rows	`bq head -n 10 DATASET.T`
Run query	`bq query --use_legacy_sql=false 'SQL'`
Dry run	`bq query --dry_run --use_legacy_sql=false 'SQL'`
Load file	`bq load --source_format=CSV DATASET.T gs://B/f.csv`
Replace on load	`bq load --replace ...`
Export table	`bq extract DATASET.T gs://B/f-*.csv`
Copy table	`bq cp [-f|-a|-n] SRC DST`
Update expiry	`bq update --expiration SECONDS DATASET.T`
Delete table / dataset	`bq rm -t DATASET.T` / `bq rm -r -d DATASET`
List / inspect jobs	`bq ls -j -n 20` / `bq show -j JOB_ID`
Cancel job	`bq cancel JOB_ID`
Wait for job	`bq wait JOB_ID 300`
Snapshot	`bq cp --snapshot SRC SNAP`
IAM bind	`bq add-iam-policy-binding --member=... --role=... DATASET.T`
Scheduled query	`bq mk --transfer_config --data_source=scheduled_query ...`
External def	`bq mkdef --source_format=PARQUET gs://B/*.parquet`